



Date: __ / __ / __

Experiment 6

6. a. Develop an efficient implementation of Floyd's algorithm to determine the shortest paths between all pairs of vertices in weighted graph. Floyd's Algorithm efficiently computes this using dynamic programming. The input should be a weighted adjacency matrix representing the graph, and the output should be the shortest distance matrix showing the shortest path distances between every pair of vertices.

Type of Lab Activity: Exercise

Aim	Material	Methods	Answer
Given	Given	Given as Algorithm	Open

Algorithm

ALGORITHM *Floyd*($W[1..n, 1..n]$)

//Implements Floyd's algorithm for the all-pairs shortest-paths problem

//Input: The weight matrix W of a graph with no negative-length cycle

//Output: The distance matrix of the shortest paths' lengths

$D \leftarrow W$ //is not necessary if W can be overwritten

for $k \leftarrow 1$ **to** n **do**

for $i \leftarrow 1$ **to** n **do**

for $j \leftarrow 1$ **to** n **do**

$D[i, j] \leftarrow \min\{D[i, j], D[i, k] + D[k, j]\}$

return D

Program

(Only of faculty reference, should not be shared to students)

```
#include <stdio.h>
```

```
#define INF 99999
```

```
#define V 4
```

```
void printSolution(int dist[][V]);
```

```
void floydWarshall(int graph[][V]) {
```

```
    int dist[V][V];
```

```
    int i, j, k;
```

```
    // Initialize the solution matrix same as input graph matrix
```

Design and Analysis of Algorithms, Laboratory Manual

Department of Computer Science and Engineering, Dayananda Sagar University, Bengaluru,
Karnataka.



```
for (i = 0; i < V; i++)
    for (j = 0; j < V; j++)
        dist[i][j] = graph[i][j];

// Add all vertices one by one to the set of intermediate vertices
for (k = 0; k < V; k++) {
    // Pick all vertices as source one by one
    for (i = 0; i < V; i++) {
        // Pick all vertices as destination for the
        // above picked source
        for (j = 0; j < V; j++) {
            // If vertex k is on the shortest path from
            // i to j, then update the value of dist[i][j]
            if (dist[i][k] + dist[k][j] < dist[i][j])
                dist[i][j] = dist[i][k] + dist[k][j];
        }
    }
}

// Print the shortest distance matrix
printSolution(dist);
}

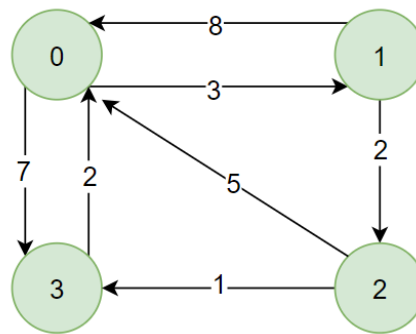
void printSolution(int dist[][V]) {
    printf("The following matrix shows the shortest distances between every pair of\nvertices:\n");
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            if (dist[i][j] == INF)
                printf("%7s", "INF");
            else
                printf("%7d", dist[i][j]);
        }
        printf("\n");
    }
}

int main() {
    int graph[V][V] = {{0, 3, INF, 7},
                       {8, 0, 2, INF},
                       {5, INF, 0, 1},
                       {2, INF, INF, 0}};

    floydWarshall(graph);
    return 0;
}
```



Sample Output



The following matrix shows the shortest distances between every pair of vertices:

0	3	5	6
5	0	2	3
3	6	0	1
2	5	7	0